```
import tkinter as tk
import tkinter.ttk as ttk
from tkinter import messagebox
import darkdetect
import firebase_admin
import re

from users.educator import Educator
from users.learner import Learner
from users.parent import Parent

cred_obj = firebase_admin.credentials.Certificate(
    "./data/codeventure-fff86-firebase-adminsdk-hwzuf-ec35e46c3c.json"
)
default_app = firebase_admin.initialize_app(
    cred_obj,
    {
        "databaseURL":
        "https://codeventure-fff86-default-rtdb.asia-southeast1.firebasedatabase.app/"
    },
)


class Theme:
    DARK = {
        "bg": "#2E2E2E",
        "fg": "#FFFFFF",
        "btn": "#424242",
        "entry_bg": "#424242",
        "entry_fg": "#FFFFFF",
    }
    LIGHT = {
        "bg": "#FFFFFF",
        "fg": "#000000",
        "btn": "#DDDDDD",
        "entry_bg": "#FFFFFF",
        "entry_fg": "#000000",
    }


class CodeVenture:
    def __init__(self):
        """
        Constructor for the CodeVenture class.
        """
        self.users = self.populate_users_data()

    def register(self):
        """
        This method is used to register a new user.
        """
        email = input("Enter email: ")
        first_name = input("Enter first name: ")
        last_name = input("Enter last name: ")
        dob = input("Enter date of birth (DD/MM/YYYY): ")
        username = input("Enter username: ")
        password = input("Enter password: ")
        confirm_password = input("Confirm password: ")
        role = input("\n1. Learner \n2. Educator \n3. Parent\nChoose your role: ")

        if password != confirm_password:
            print("Passwords do not match.")
            return

        if (
            not email
            or not first_name
            or not last_name
            or not dob
            or not username
            or not password
            or not confirm_password
            or not role
```

```
            ):
                print("Please fill in all the details.")
                return

            if username in [user.get_username() for user in self.users]:
                print("Username already exists.")
                return

            # Creating a new user object depending on the role, and adding it to the
            users list
            if role == "1":
                user = Learner(email, first_name, last_name, username, password, dob)
            elif role == "2":
                user = Educator(email, first_name, last_name, username, password, dob)
            elif role == "3":
                user = Parent(email, first_name, last_name, username, password, dob)
            else:
                print("Please choose a valid role.")
                return

            # Saving the user details to the users.txt file
            with open("./data/users.txt", "a") as file:
                user_string = (
                    f"{first_name},{last_name},{dob},{username},{password},{email}\n"
                )
                file.write(user_string)

            self.users.append(user)
            print(f"User {first_name} {last_name} registered successfully!")

        def login(self):
            """
            This method is used to login a user.
            """
            username = input("Enter username: ")
            password = input("Enter password: ")

            for user in self.users:
                if user.get_username() == username and user.get_password() == password:
                    user.dashboard()
                    break
                else:
                    print("Invalid username or password.")

        def populate_users_data(self, path="./data/users.txt"):
            """
            Populate users data into program memory.

            :param path: The path to the users data file.
            """
            users_data = []

            try:
                file = open(path, "r", encoding="utf8")
                lines = list(file)
                for line in lines:
                    (
                        first_name,
                        last_name,
                        dob,
                        username,
                        password,
                        email,
                        role,
                    ) = line.strip("\n").split(",")

                    if role == "Learner":
                        user = Learner(
                            email, first_name, last_name, username, password, dob
                        )
                    elif role == "Educator":
                        user = Educator(
                            email, first_name, last_name, username, password, dob
```

```
                    )
                elif role == "Parent":
                    user = Parent(email, first_name, last_name, username, password,
                    dob)

                users_data.append(user)
            file.close()
        except:
            raise FileNotFoundError("The users data file does not exist!")
        finally:
            return users_data


class CodeVentureGUI:
    def __init__(self, master, app):
        """
        Constructor for the CodeVentureGUI class.

        :param master: The parent widget.
        :param app: The CodeVenture object.
        """
        self.master = master
        self.app = app
        self.theme = Theme.DARK if darkdetect.isDark() else Theme.LIGHT

        master.title("CodeVenture")

        self.label_title = tk.Label(master, text="CodeVenture", font=("Arial", 24))
        self.label_title.pack(pady=20)

        self.login_button = tk.Button(master, text="Login", command=self.login_screen)
        self.login_button.pack(pady=20)

        self.register_button = tk.Button(
            master, text="Register", command=self.register_screen
        )
        self.register_button.pack(pady=20)

        self.theme_button = tk.Button(
            master, text="Toggle Dark Mode", command=self.toggle_theme
        )
        self.theme_button.pack(pady=20)

    def toggle_theme(self):
        """
        This method is used to toggle the theme of the application.
        """
        self.theme = Theme.DARK if self.theme == Theme.LIGHT else Theme.LIGHT
        self.master.configure(bg=self.theme["bg"])
        self.label_title.configure(bg=self.theme["bg"], fg=self.theme["fg"])
        self.login_button.configure(bg=self.theme["btn"], fg=self.theme["fg"])
        self.register_button.configure(bg=self.theme["btn"], fg=self.theme["fg"])
        self.theme_button.configure(bg=self.theme["btn"], fg=self.theme["fg"])

    def login_screen(self):
        """
        This method is used to display the login screen.
        """
        # self.master.withdraw()  # Hide the main window
        self.new_window = tk.Toplevel(self.master)
        self.login_app = Login(self.new_window, self.app, self.master)

    def register_screen(self):
        """
        This method is used to display the register screen.
        """
        self.new_window = tk.Toplevel(self.master)
        self.register_app = Register(self.new_window, self.app)


class Login:
    def __init__(self, master, app, main_window):
        """
```

```
217             Constructor for the Login class.
218
219             :param master: The parent widget.
220             :param app: The CodeVenture object.
221             :param main_window: The main window of the application.
222             """
223             self.theme = Theme.LIGHT
224             self.master = master
225             self.app = app
226             self.main_window = main_window
227
228             master.title("Login")
229
230             self.label_username = tk.Label(master, text="Username")
231             self.label_username.pack(pady=10)
232
233             self.entry_username = tk.Entry(master)
234             self.entry_username.pack(pady=10)
235
236             self.label_password = tk.Label(master, text="Password")
237             self.label_password.pack(pady=10)
238
239             self.entry_password = tk.Entry(master, show="*")
240             self.entry_password.pack(pady=10)
241
242             self.login_button = tk.Button(master, text="Login",
                command=self.validate_login)
243             self.login_button.pack(pady=20)
244
245             self.theme_button = tk.Button(
246                 master, text="Toggle Dark Mode", command=self.toggle_theme
247             )
248             self.theme_button.pack(pady=20)
249
250         def toggle_theme(self):
251             """
252             This method is used to toggle the theme of the application.
253             """
254             self.theme = Theme.DARK if self.theme == Theme.LIGHT else Theme.LIGHT
255             self.master.configure(bg=self.theme["bg"])
256             self.label_username.configure(bg=self.theme["bg"], fg=self.theme["fg"])
257             self.label_password.configure(bg=self.theme["bg"], fg=self.theme["fg"])
258             self.login_button.configure(bg=self.theme["btn"], fg=self.theme["fg"])
259             self.theme_button.configure(bg=self.theme["btn"], fg=self.theme["fg"])
260
261         def validate_login(self):
262             """
263             This method is used to validate the login details.
264             """
265             # Validate the login details
266             username = self.entry_username.get()
267             password = self.entry_password.get()
268
269             for user in self.app.users:
270                 if user.get_username() == username and user.get_password() == password:
271                     messagebox.showinfo("Info", "Login Successful!")
272                     user.dashboard(self.master)
273                     return
274             messagebox.showerror("Error", "Invalid credentials!")
275
276
277     class Register:
278         def __init__(self, master, app):
279             """
280             Constructor for the Register class.
281
282             :param master: The parent widget.
283             :param app: The CodeVenture object.
284             """
285             self.theme = Theme.LIGHT
286             self.master = master
287             self.app = app
288             master.title("Register")
```

```
289
290         # Initialize widgets
291         self.initialize_firstname()
292         self.initialize_lastname()
293         self.initialize_dob()
294         self.initialize_email()
295         self.initialize_username()
296         self.initialize_password()
297         self.initialize_confirm_password()
298         self.initialize_role()
299         self.initialize_buttons()
300
301         self.theme_button = tk.Button(
302             master, text="Toggle Dark Mode", command=self.toggle_theme
303         )
304         self.theme_button.pack(pady=20)
305
306     def initialize_firstname(self):
307         """
308         This method is used to initialize the first name widgets.
309         """
310         self.label_firstname = tk.Label(self.master, text="First Name")
311         self.label_firstname.pack(pady=10)
312         self.entry_firstname = tk.Entry(self.master)
313         self.entry_firstname.pack(pady=10)
314
315     def initialize_lastname(self):
316         """
317         This method is used to initialize the last name widgets.
318         """
319         self.label_lastname = tk.Label(self.master, text="Last Name")
320         self.label_lastname.pack(pady=10)
321         self.entry_lastname = tk.Entry(self.master)
322         self.entry_lastname.pack(pady=10)
323
324     def initialize_dob(self):
325         """
326         This method is used to initialize the date of birth widgets.
327         """
328         self.label_dob = tk.Label(self.master, text="Date of Birth")
329         self.label_dob.pack(pady=10)
330         self.dob_frame = tk.Frame(self.master)
331         self.dob_frame.pack(pady=10)
332
333         self.day_var = tk.StringVar()
334         self.month_var = tk.StringVar()
335         self.year_var = tk.StringVar()
336
337         combobox_configurations = [
338             ("entry_day", self.day_var, 2, range(1, 32)),
339             ("entry_month", self.month_var, 2, range(1, 13)),
340             ("entry_year", self.year_var, 4, range(1900, 2025)),
341         ]
342
343         for widget_name, var, width, values in combobox_configurations:
344             setattr(
345                 self,
346                 widget_name,
347                 ttk.Combobox(
348                     self.dob_frame,
349                     textvariable=var,
350                     width=width,
351                     values=[str(i).zfill(width) for i in values],
352                 ),
353             )
354             getattr(self, widget_name).pack(side="left", padx=5)
355
356     def initialize_email(self):
357         """
358         This method is used to initialize the email widgets.
359         """
360         self.label_email = tk.Label(self.master, text="Email Address")
361         self.label_email.pack(pady=10)
```

```
362             self.entry_email = tk.Entry(self.master)
363             self.entry_email.pack(pady=10)
364
365         def initialize_username(self):
366             """
367             This method is used to initialize the username widgets.
368             """
369             self.label_username = tk.Label(self.master, text="Username")
370             self.label_username.pack(pady=10)
371             self.entry_username = tk.Entry(self.master)
372             self.entry_username.pack(pady=10)
373
374         def initialize_password(self):
375             """
376             This method is used to initialize the password widgets.
377             """
378             self.label_password = tk.Label(self.master, text="Password")
379             self.label_password.pack(pady=10)
380             self.entry_password = tk.Entry(self.master, show="*")
381             self.entry_password.pack(pady=10)
382
383         def initialize_confirm_password(self):
384             """
385             This method is used to initialize the confirm password widgets.
386             """
387             self.label_confirm_password = tk.Label(self.master, text="Confirm Password")
388             self.label_confirm_password.pack(pady=10)
389             self.entry_confirm_password = tk.Entry(self.master, show="*")
390             self.entry_confirm_password.pack(pady=10)
391
392         def initialize_role(self):
393             """
394             This method is used to initialize the role widgets.
395             """
396             self.label_role = tk.Label(self.master, text="Role")
397             self.label_role.pack(pady=10)
398             self.role_options = ["Learner", "Educator", "Parent"]
399             self.role_var = tk.StringVar()
400             self.role_combobox = ttk.Combobox(
401                 self.master, textvariable=self.role_var, values=self.role_options
402             )
403             self.role_combobox.set(self.role_options[0])
404             self.role_combobox.pack(pady=10)
405
406         def initialize_register_button(self):
407             """
408             This method is used to initialize the register button.
409             """
410             self.register_button = tk.Button(
411                 self.master, text="Register", command=self.validate_register
412             )
413             self.register_button.pack(pady=20)
414
415         def validate_email(self, email: str) -> bool:
416             """
417             This method is used to validate the email address.
418
419             :param email: The email address to be validated.
420             """
421             # regex for email validation:
422             pattern = r"^[a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-.]+$"
423             return bool(re.match(pattern, email))
424
425         # for simplicity, password validation is only checking for length
426         def validate_password(self, password: str) -> bool:
427             """
428             This method is used to validate the password.
429
430             :param password: The password to be validated.
431             """
432             if len(password) < 4:
433                 return False
434             return True
```

```
435
436     def validate_register(self):
437         """
438         This method is used to validate the registration details.
439         """
440         # Get the date in the format you mentioned
441         dob = f"{self.day_var.get()}/{self.month_var.get()}/{self.year_var.get()}"
442         if (
443             not self.entry_firstname.get()
444             or not self.entry_lastname.get()
445             or not self.entry_password.get()
446             or not dob
447             or not self.entry_email.get()
448         ):
449             messagebox.showerror("Error", "Please fill in all the required fields!")
450             return
451
452         reg_username = self.entry_username.get()
453         reg_password = self.entry_password.get()
454         reg_confirm_password = self.entry_confirm_password.get()
455         reg_firstname = self.entry_firstname.get()
456         reg_lastname = self.entry_lastname.get()
457         reg_email = self.entry_email.get()
458         reg_role = self.role_var.get()
459         reg_dob = dob
460
461         if not reg_firstname.isalpha() or not reg_lastname.isalpha():
462             messagebox.showerror("Error", "Name should only contain alphabets!")
463             return
464
465         if reg_username in [user.get_username() for user in self.app.users]:
466             messagebox.showerror("Error", "Username already exists!")
467             return
468
469         # calling the email validation function
470         if not self.validate_email(reg_email):
471             messagebox.showerror("Error", "Please enter a valid email!")
472             return
473
474         # calling the password validation function
475         if not self.validate_password(reg_password):
476             messagebox.showerror(
477                 "Error", "Password cannot have less than 4 characters!"
478             )
479             return
480
481         if reg_dob == "01/01/1900":
482             messagebox.showerror("Error", "Please enter a valid date of birth!")
483             return
484
485         if reg_password != reg_confirm_password:
486             messagebox.showerror("Error", "Passwords do not match!")
487             return
488
489         with open("./data/users.txt", "a") as file:
490             user_string =
                f"{reg_firstname},{reg_lastname},{reg_dob},{reg_username},{reg_password},{
                reg_email},{reg_role}\n"
491             file.write(user_string)
492
493         messagebox.showinfo(
494             "Registration Successful!",
495             "You have been registered successfully! Please login to continue.",
496         )
497
498     def toggle_theme(self):
499         """
500         This method is used to toggle the theme of the application.
501         """
502         self.theme = Theme.DARK if self.theme == Theme.LIGHT else Theme.LIGHT
503         self.master.configure(bg=self.theme["bg"])
504         self.label_username.configure(bg=self.theme["bg"], fg=self.theme["fg"])
505         self.label_password.configure(bg=self.theme["bg"], fg=self.theme["fg"])
```

```
            self.login_button.configure(bg=self.theme["btn"], fg=self.theme["fg"])
            self.theme_button.configure(bg=self.theme["btn"], fg=self.theme["fg"])


    if __name__ == "__main__":
        app_logic = CodeVenture()
        root = tk.Tk()
        app_gui = CodeVentureGUI(root, app_logic)
        root.mainloop()

```